

데이터시각화

In [36]:

```
'''
Seaborn은 파이썬의 데이터 시각화 라이브러리 중 하나로, Matplotlib에 기반을 두고 있습니다.
주로 통계 데이터를 시각화하는 데 사용되며, Matplotlib보다 좀 더 간편하고 세련된 시각화를 제공합니다.

Seaborn은 다양한 종류의 그래프를 그릴 수 있습니다.
예를 들어, 선 그래프, 산점도, 히스토그램, 박스 플롯, 히트맵 등을 지원합니다.
또한 테마와 색상 팔레트를 손쉽게 변경하여 시각화의 스타일을 바꿀 수 있습니다.

Seaborn의 주요 특징은 다음과 같습니다:

1. 통계적 시각화: 데이터를 분석하고 시각화할 때 통계적 기법을 활용하여
   보다 의미 있는 그래프를 생성할 수 있습니다.
2. 높은 수준의 추상화: Seaborn은 Matplotlib을 기반으로 하지만,
   보다 높은 수준의 추상화를 제공하여 복잡한 그래프를 간단하게 그릴 수 있습니다.
3. 기본적으로 세련된 스타일: Seaborn은 기본적으로 시각적으로 매력적인 스타일을 제공하여
   플롯을 더 깔끔하고 세련되게 만들어 줍니다.
4. 편리한 데이터 구조 지원: Seaborn은 데이터프레임 형태의 데이터를 쉽게 다룰 수 있습니다.
   또한 범주형 데이터의 시각화를 지원하여 카테고리에 따른 차이를 쉽게 파악할 수 있습니다.

Seaborn은 주로 데이터 분석 및 시각화에 사용되며, 데이터의 분포, 관계, 패턴 등을 탐색하고
시각적으로 전달하는 데 유용합니다. Matplotlib과 함께 사용되어
데이터 과학 및 머신러닝 프로젝트에서 널리 활용됩니다.
'''

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image
```

Violin Plot

In [2]:

```
'''
Violin plot은 데이터의 분포를 시각화하는데 사용되는 그래프 형식 중 하나입니다.
Seaborn 패키지에서 제공되며, 박스 플롯(Box plot)과
커널 밀도 플롯(Kernel density plot)의 장점을 결합한 형태입니다.
주로 연속형 변수의 분포를 비교하고 시각화하는 데 유용합니다.

Violin plot은 다음과 같은 주요 특징을 갖습니다:
1. 박스 플롯과 유사한 요약 통계 정보 제공: Violin plot은 박스 플롯처럼
   데이터의 중앙값, 사분위수, 이상치 등을 시각적으로 제공합니다.
2. 커널 밀도 추정: Violin plot은 밀도 추정(kernel density estimation)을 사용하여
   데이터의 분포를 부드럽게 나타냅니다. 각각의 바이올린 형태는 해당 데이터의 밀도를 나타냅니다.
   더 두꺼운 부분은 데이터가 많이 분포된 곳을 나타냅니다.
3. 다중 그룹 비교: Violin plot을 사용하면 여러 그룹의 데이터 분포를 한 번에 비교할 수 있습니다.
   이는 그룹 간의 차이를 시각적으로 파악하는 데 유용합니다.
4. 적절한 스케일링: Violin plot은 데이터의 분포를 제대로 표현하기 위해
   자동으로 스케일링을 수행합니다.
   이는 데이터의 범위가 다른 경우에도 적절한 비교를 가능하게 합니다.

Violin plot은 데이터의 분포를 전달하는 데 매우 효과적이며,
이상치 및 중앙값과 같은 요약 통계 정보를 함께 제공하여
데이터의 특성을 잘 이해할 수 있습니다.
특히 데이터가 많은 경우에도 밀도를 잘 표현하여 유용한 시각화를 제공합니다.

Violin plot은 다음과 같은 주요 구성 요소를 갖습니다:

1. 바이올린 형태(Violin shape):
   - Violin plot의 주된 부분으로, 데이터의 분포를 보여줍니다.
   - 각 바이올린은 커널 밀도 추정을 기반으로 그려져 데이터의 분포를 나타냅니다.
   - 바이올린의 폭은 데이터의 밀도를 나타내며,
     데이터가 많이 분포된 부분은 더 넓게 표현됩니다.

2. 중앙 선(Median line):
   - Violin plot 안의 중앙에 그려지는 선으로, 데이터의 중앙값을 나타냅니다.
   - 이 선은 바이올린 형태를 따라 그려집니다.

3. 상자(Box):
   - 중앙 선을 둘러싸고 있는 상자 모양의 부분입니다.
   - 상자의 위쪽과 아래쪽은 각각 상위 사분위수(75th percentile)와
     하위 사분위수(25th percentile)를 나타냅니다.
'''
```

- 상자 내부의 가로선은 중앙값(**median**)을 나타냅니다.

4. 최대/최소값을 나타내는 선(**Whiskers**):

- 상자의 위 아래에 그려지는 선으로, 데이터의 최댓값과 최솟값을 나타냅니다.
- 일반적으로, 이 선은 데이터의 1.5배 사분위 범위(**IQR, Interquartile Range**)까지만 그려집니다. 이 범위를 넘어가는 데이터는 이상치(**outlier**)로 간주됩니다.

5. 점(**Outliers**):

- 상자 밖에 그려지는 개별 데이터 포인트로, 주어진 데이터 집합에서 다른 값들과 크게 다른 값들을 나타냅니다.
- 이상치는 일반적으로 **Whiskers**에서 멀리 떨어진 위치에 있습니다.

이러한 구성 요소들은 데이터의 분포, 중앙값, 사분위수, 이상치 등을 시각적으로 파악할 수 있도록 도와줍니다.

이를 통해 데이터의 특성을 빠르게 이해하고 비교할 수 있습니다.

'''

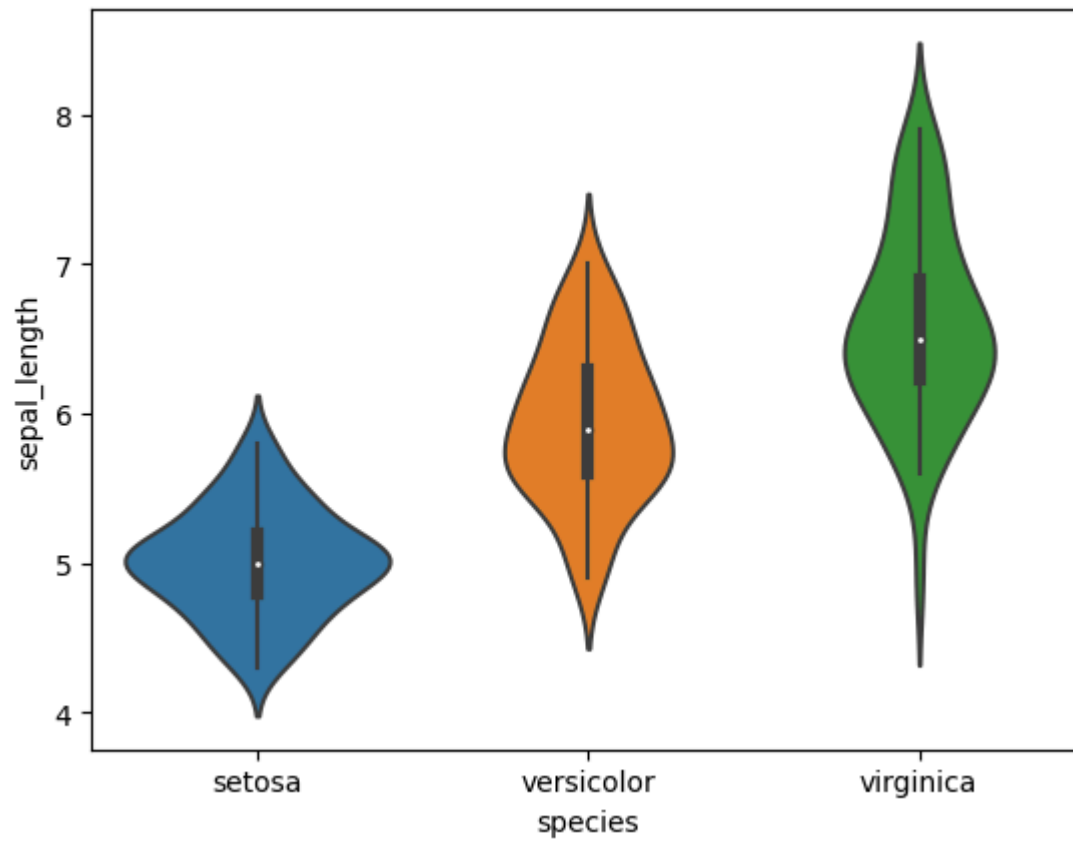
```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("iris")
print(data.columns)

# Violin plot 그리기
sns.violinplot(x="species", y="sepal_length", data=data)

# 그래프 보여주기
plt.show()
```

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
      'species'],
      dtype='object')
```



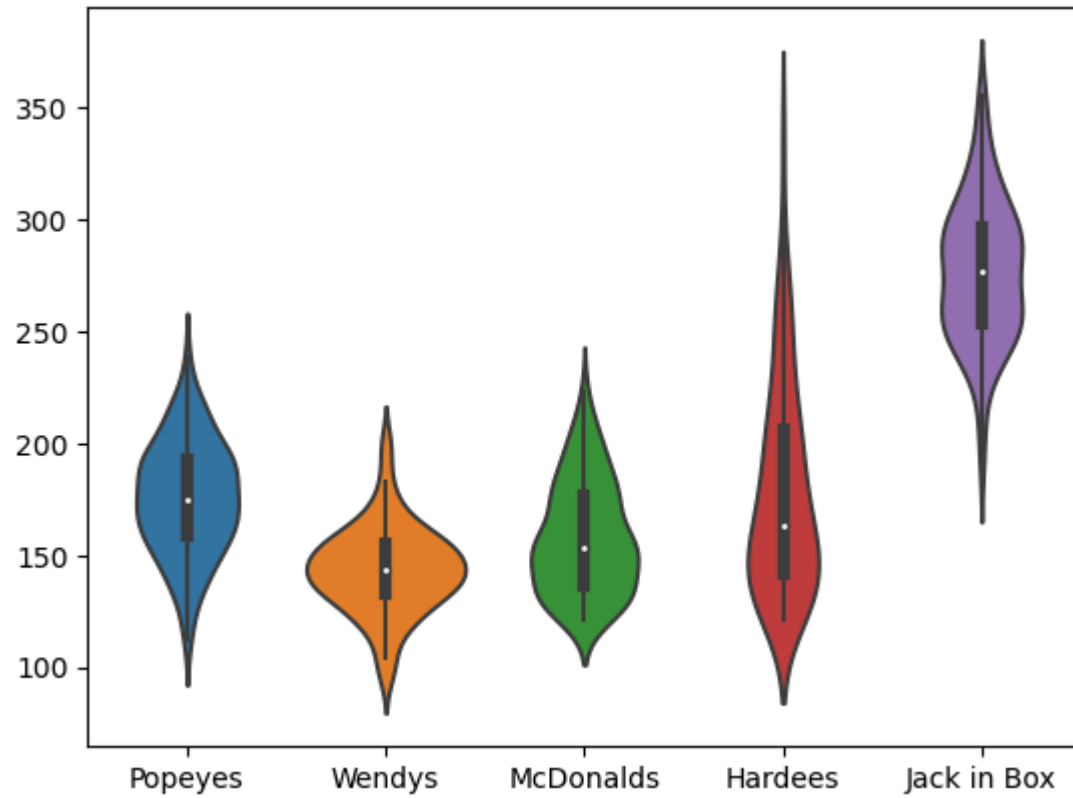
```
In [3]: fastfood = pd.read_csv("fastfood.csv")
fastfood
```

Out[3]:

	Popeyes	Wendys	McDonalds	Hardees	Jack in Box
0	150	173	198	158	289
1	197	95	131	195	273
2	186	154	158	177	254
3	166	104	123	147	282
4	196	183	139	181	235
...	...	...	...	...	...
95	195	138	145	235	272
96	213	154	178	218	261
97	135	178	133	147	271
98	136	152	124	154	263
99	205	119	180	202	265

100 rows × 5 columns

In [4]: `#pd.DataFrame.iteritems = pd.DataFrame.items`In [5]: `sns.violinplot(data=fastfood)
plt.show()`



```
In [6]: iris = pd.read_csv("iris.csv")  
iris
```

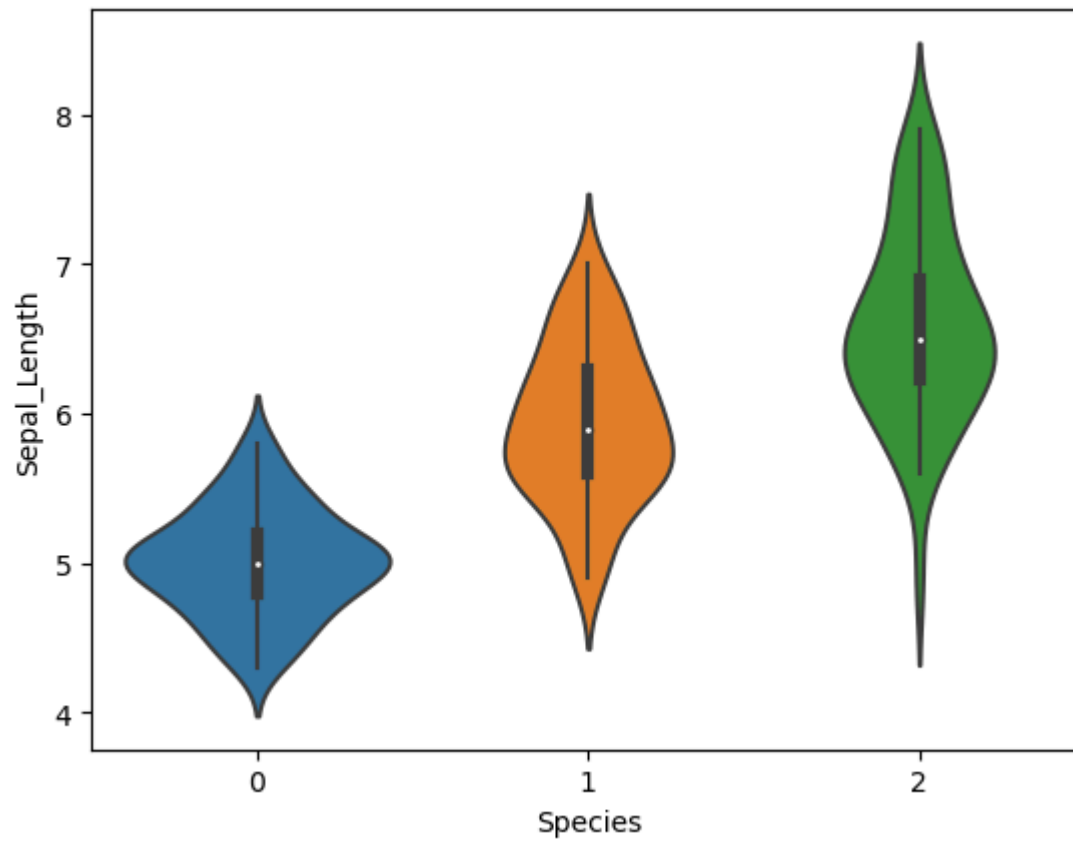
Out[6]:

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

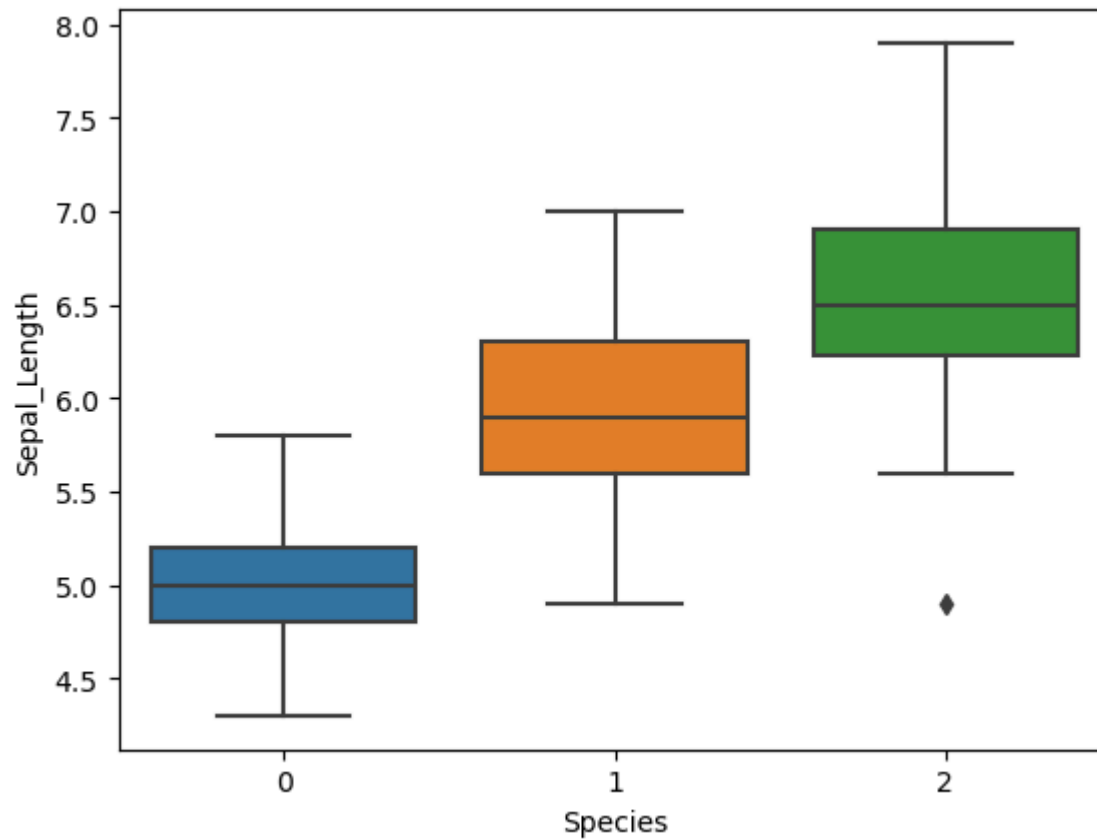
```
In [7]: sns.violinplot(data=iris, y="Sepal_Length", x="Species")
        #plt.show()
```

```
Out[7]: <Axes: xlabel='Species', ylabel='Sepal_Length'>
```



```
In [8]: sns.boxplot(data=iris, y="Sepal_Length", x="Species")
```

```
Out[8]: <Axes: xlabel='Species', ylabel='Sepal_Length'>
```

Stem and Leaf Plot

```
In [ ]: #!pip install stemgraphic
```

```
In [10]: #!pip install --upgrade pandas "dask[complete]"
```

```
In [11]: '''
stemgraphic은 숫자 데이터의 분포를 시각화하기 위한 Python 패키지 중 하나입니다.
일반적으로 히스토그램과 비슷하지만, 히스토그램보다 더 간단하고 직관적인 방식으로
데이터의 분포를 표현할 수 있습니다.
```

주요 특징과 사용법은 다음과 같습니다:

1. 간단하고 직관적인 시각화: **stemgraphic**는 매우 간단하며 직관적인 방식으로 데이터의 분포를 시각화합니다. 데이터의 분포를 한눈에 파악할 수 있습니다.
2. 스템-리프 플롯(**Stem-and-Leaf Plot**): **stemgraphic**은 주로 스템-리프 플롯을 생성하는 데 사용됩니다. 이는 각 데이터 포인트의 십의 자리와 일의 자리를 스템(**stem**)과 리프(**leaf**)로 나누어 나타내는 방식입니다.
3. 숫자 데이터 시각화: 주로 숫자 데이터의 분포를 시각화하는 데 사용됩니다. 이는 통계적 분석이나 데이터 탐색을 위한 초기 단계에서 유용합니다.
4. **matplotlib**와의 통합: **stemgraphic**은 **matplotlib**와 통합되어 있으며, 생성된 플롯을 필요에 따라 사용자 정의하거나 다른 **matplotlib** 그래픽과 결합할 수 있습니다.
5. 다양한 옵션: **stemgraphic**은 여러 가지 옵션을 제공하여 사용자가 원하는 방식으로 플롯을 조정할 수 있습니다. 예를 들어, 스템의 길이, 간격, 숫자 포맷 등을 설정할 수 있습니다.

사용법은 다음과 같습니다:

```
import stemgraphic

# 데이터 준비
data = [10, 15, 20, 25, 30, 35, 40, 45, 50]

# 스템-리프 플롯 생성
stemgraphic.stem_graphic(data)

# 스템-리프 플롯을 파일로 저장
stemgraphic.stem_graphic(data, scale=10, filename='stem_leaf_plot.png')

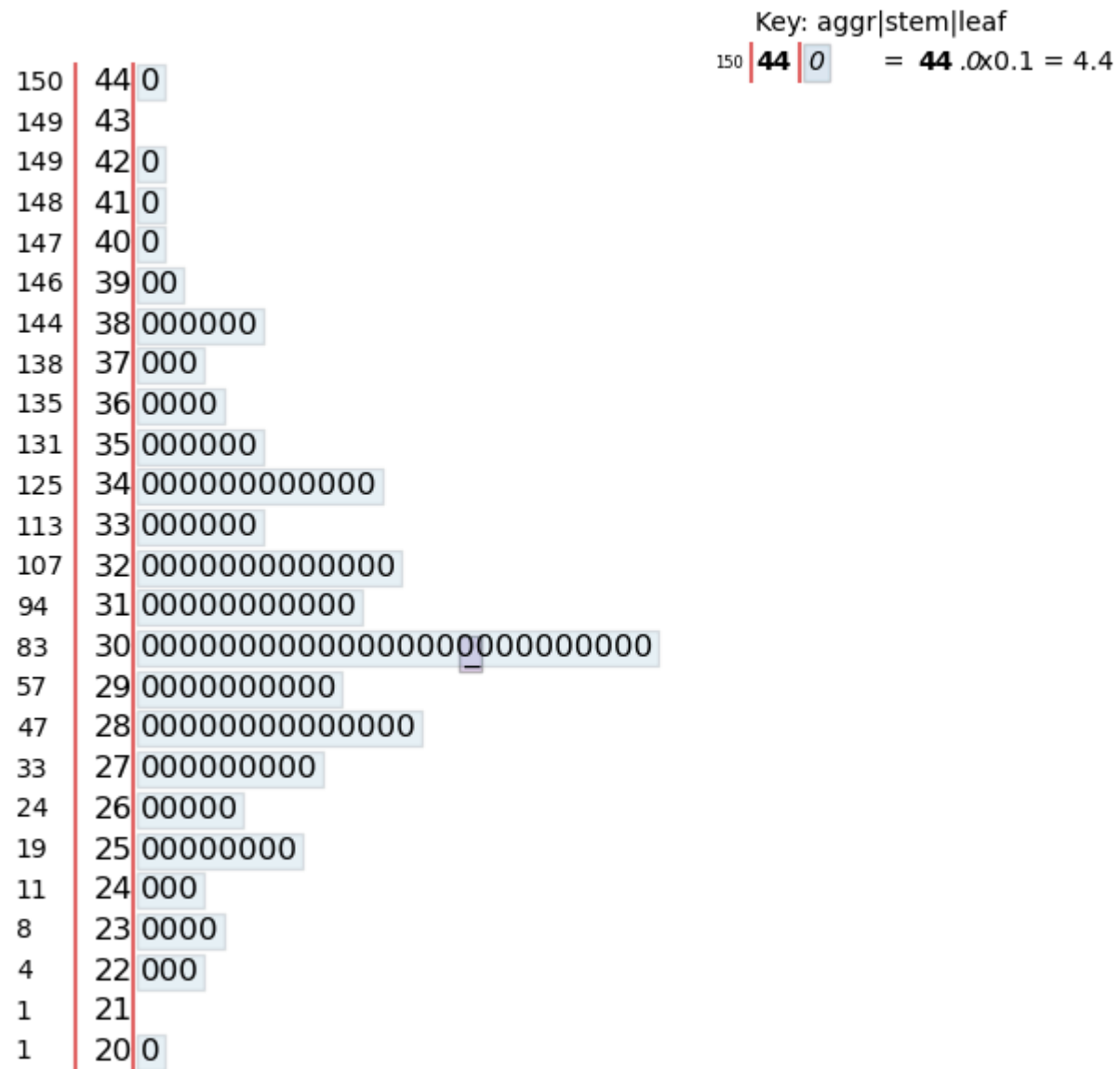
위 코드는 데이터 리스트를 사용하여 기본적인 스템-리프 플롯을 생성하고,
또한 플롯을 이미지 파일로 저장하는 방법을 보여줍니다.
'''

import stemgraphic
```

```
In [12]: iris['Sepal_Width']
```

```
Out[12]: 0      3.5  
        1      3.0  
        2      3.2  
        3      3.1  
        4      3.6  
        ...  
       145     3.0  
       146     2.5  
       147     3.0  
       148     3.4  
       149     3.0  
Name: Sepal_Width, Length: 150, dtype: float64
```

```
In [13]: stemgraphic.stem_graphic(iris['Sepal_Width'])  
plt.show()
```



Parallel Coordinates Plot

```
In [14]: '''
`pandas.plotting.parallel_coordinates()` 함수는 병렬 좌표계(parallel coordinates)를
사용하여 다차원 데이터를 시각화하는 데 사용됩니다.
각 열이 데이터의 차원을 나타내며, 각 행이 데이터 포인트를 나타냅니다.
이 그래프를 사용하면 각 열(차원)마다의 데이터 분포 및 패턴을 비교할 수 있습니다.

이 함수의 주요 특징은 다음과 같습니다:
1. 다차원 데이터 시각화: 병렬 좌표계를 사용하여 다차원 데이터를 시각화합니다.
   각 열은 데이터의 차원을 나타내며, 각 행은 데이터 포인트를 나타냅니다.
   이를 통해 다차원 데이터의 패턴과 관계를 한눈에 파악할 수 있습니다.
2. 열(차원) 간 비교: 각 열(차원)을 평행선으로 나타내고,
   데이터 포인트의 선이 이 평행선들을 횡단하면서 데이터의 값 변화를 보여줍니다.
   이를 통해 각 열 간의 상관 관계나 패턴을 시각적으로 파악할 수 있습니다.
3. 클래스 구분: 클래스 열을 제공하면 데이터 포인트를 클래스에 따라
   색으로 구분하여 시각화할 수 있습니다.
   이를 통해 각 클래스 간의 차이를 쉽게 비교할 수 있습니다.
4. 간단하고 직관적: 사용법이 간단하고 직관적이며, Pandas의 DataFrame을 기반으로 하므로
   데이터프레임의 데이터를 바로 시각화할 수 있습니다.
5. Matplotlib 통합: Matplotlib를 사용하여 그래프를 생성하므로,
   다른 Matplotlib 함수와 함께 사용하여 그래프를 수정하거나 다른 그래픽 요소를 추가할 수 있습니다.
6. 추가적인 설정: 다양한 옵션을 제공하여 그래프의 선 두께, 색상, 투명도 등을 조절할 수 있습니다.

이러한 특징들은 `pandas.plotting.parallel_coordinates()` 함수를 사용하여
다차원 데이터의 분포와 패턴을 시각화하는 데 유용하게 활용할 수 있습니다.

이 함수의 주요 인자와 기능은 다음과 같습니다:
1. frame (DataFrame):
   - 시각화할 데이터를 포함하는 Pandas의 DataFrame 객체입니다.
2. class_column (str, optional):
   - 데이터프레임에서 클래스를 식별하는 열의 이름입니다.
   이 인자를 제공하면 데이터 포인트가 클래스에 따라 색으로 구분됩니다.
3. cols (sequence, optional):
   - 시각화할 열(차원)의 이름이 포함된 순서열(Sequence)입니다.
   이 인자를 제공하지 않으면 DataFrame의 모든 숫자형 열이 시각화됩니다.
4. ax (matplotlib.axes.Axes, optional):
   - 그래프가 그려질 Matplotlib의 축(Axes) 객체입니다.
   이 인자를 제공하지 않으면 새로운 축이 생성됩니다.
5. color (str or sequence, optional):
   - 클래스 열을 기준으로 그룹화할 때 사용할 색상입니다.
```

문자열일 경우는 단일 색상을, 순서열일 경우는 각 클래스에 대한 색상을 지정합니다.

6. `use_columns (bool, optional):`

- 열 이름을 사용하여 열을 선택할지 여부를 결정합니다. 기본값은 `True`입니다. `False`로 설정하면 열을 순서대로 선택하게 됩니다.

7. `linewidth (float, optional):`

- 선의 두께를 지정합니다.

8. `alpha (float, optional):`

- 그래프의 투명도를 지정합니다. 0에서 1 사이의 값을 가집니다.

9. `axvlines (bool, optional):`

- 각 열(차원) 사이에 수직선을 그릴지 여부를 결정합니다. 기본값은 `True`입니다.

10. `sort_labels (bool, optional):`

- 클래스 레이블을 알파벳 순서로 정렬할지 여부를 결정합니다. 기본값은 `True`입니다.

이 함수를 사용하여 병렬 좌표계를 그리면 다차원 데이터의 패턴과 관계를 시각적으로 파악할 수 있습니다. 데이터셋의 차원이 많을 때 특히 유용하며, 클래스 열을 제공하면 데이터 포인트를 클래스에 따라 색으로 구분하여 시각화할 수 있습니다.

```
import pandas as pd
from pandas.plotting import parallel_coordinates
import matplotlib.pyplot as plt

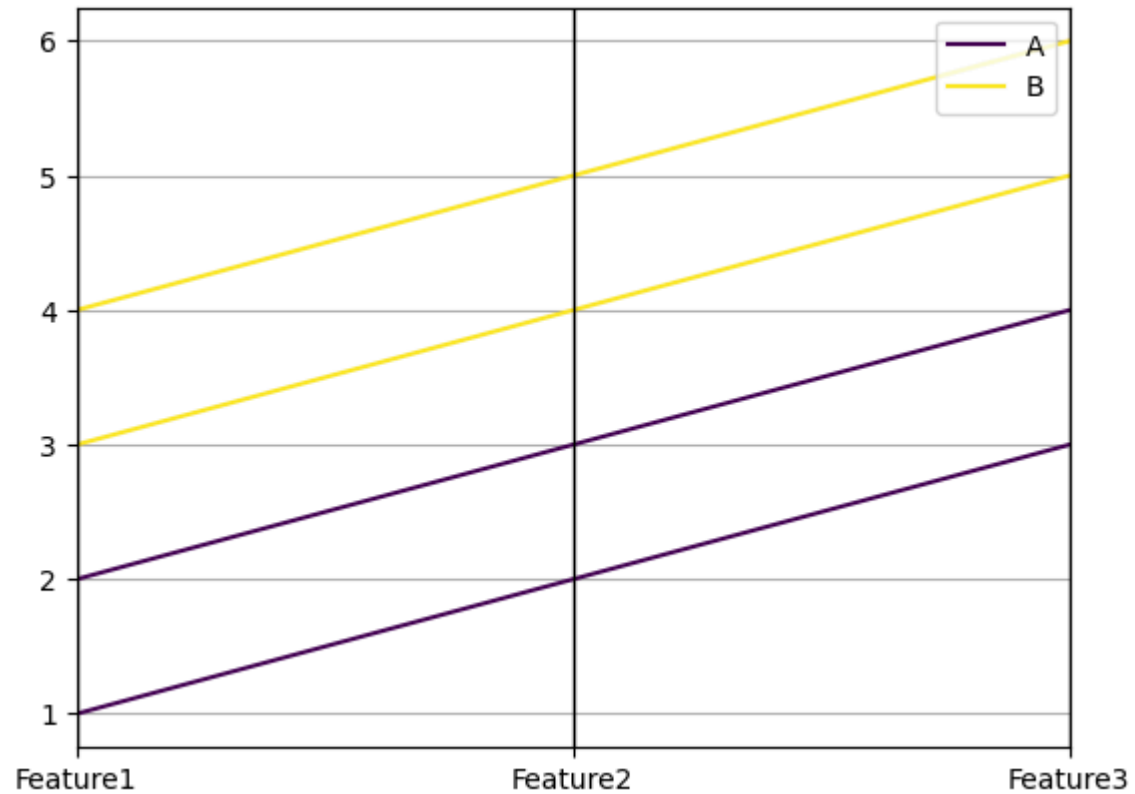
# 예시 데이터 생성
data = pd.DataFrame({
    'Feature1': [1, 2, 3, 4],
    'Feature2': [2, 3, 4, 5],
    'Feature3': [3, 4, 5, 6],
    'Class': ['A', 'A', 'B', 'B']
})

print(data)

# 병렬 좌표 차트 생성 - Class 열은 각 데이터 포인트의 클래스를 나타내며
# 이를 기준으로 색상을 구분하여 시각화한다.
parallel_coordinates(data, class_column = 'Class', colormap='viridis')

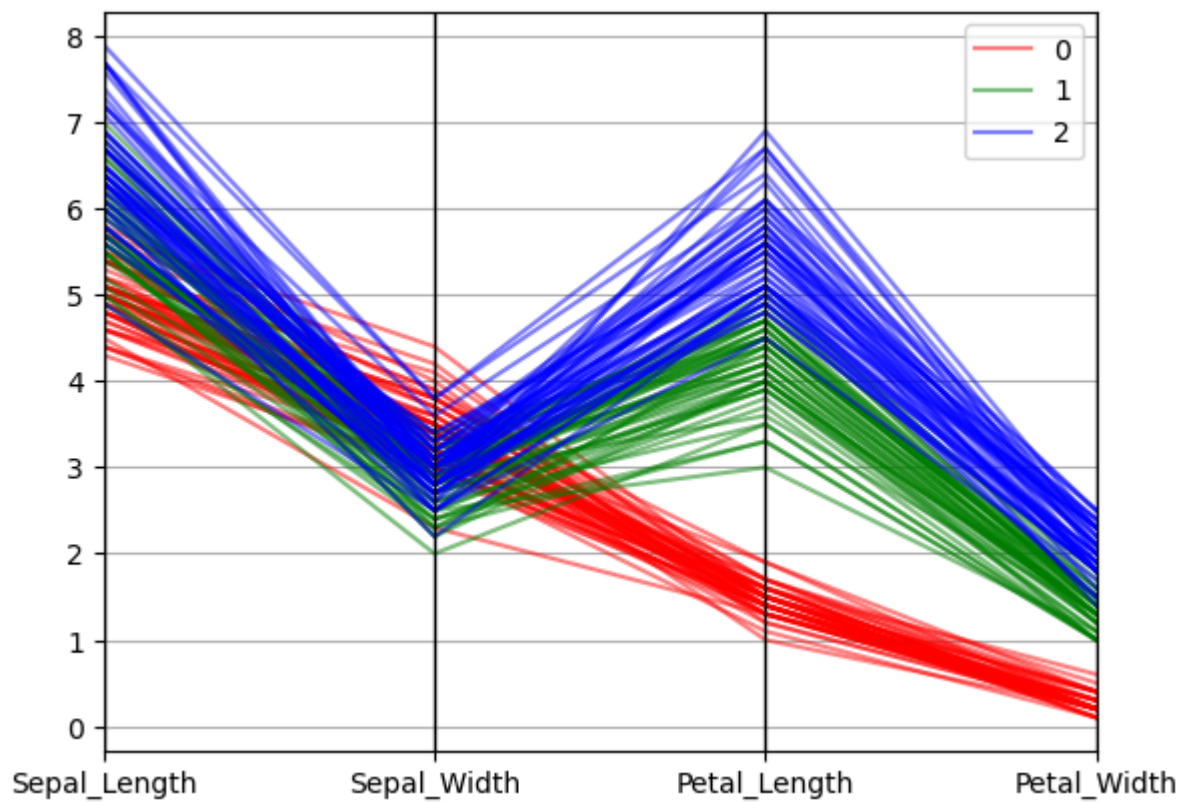
# 그래프 출력
plt.show()
```

	Feature1	Feature2	Feature3	Class
0	1	2	3	A
1	2	3	4	A
2	3	4	5	B
3	4	5	6	B



```
In [15]: from pandas.plotting import parallel_coordinates

# parallel coordinates 그리기(정규화 전)
parallel_coordinates(iris, 'Species', color=('r','g','b'), alpha=0.5)
plt.show()
```



```
In [16]: iris
```


Out[16]:

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

```
In [17]: from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df1 = iris.drop('Species', axis=1)
df1
```

Out[17]:

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

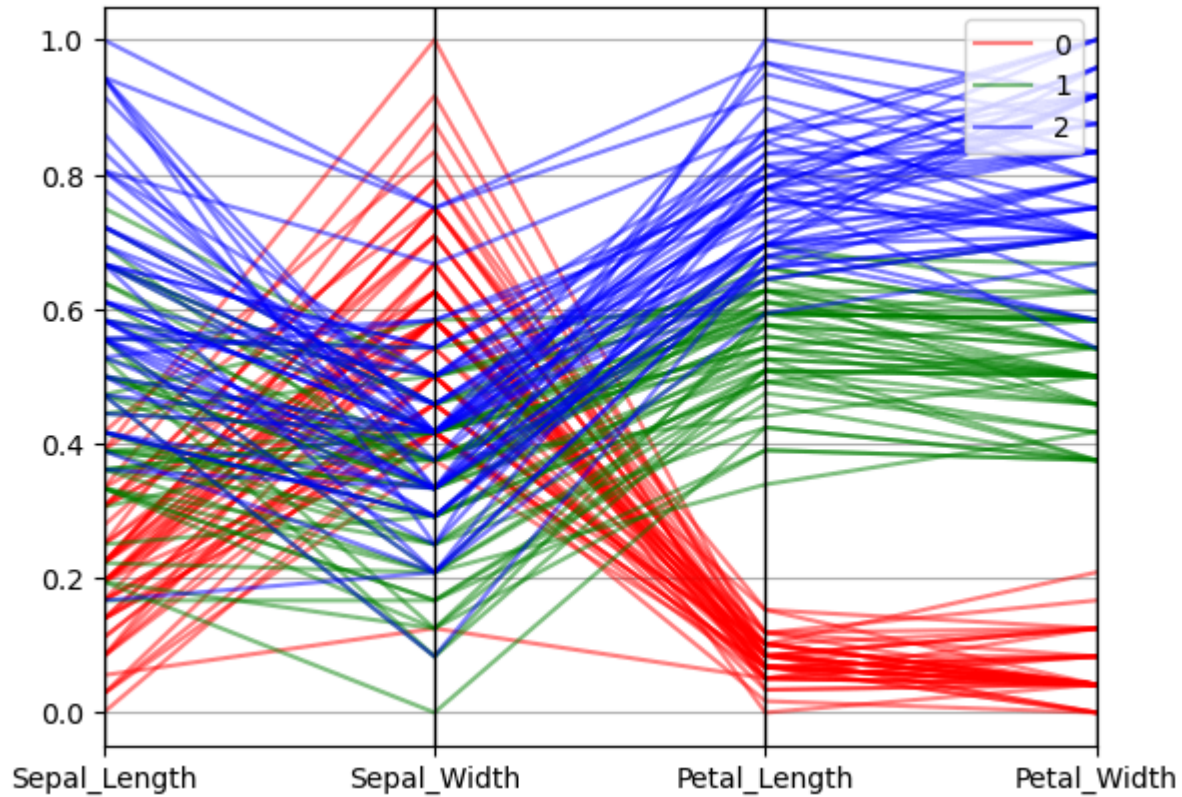
```
In [18]: df1_minmax = pd.DataFrame( scaler.fit_transform(df1) )
iris_scale = pd.concat([df1_minmax, iris['Species']], axis=1)
iris_scale.columns = iris.columns
iris_scale
```

Out[18]:

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Species
0	0.222222	0.625000	0.067797	0.041667	0
1	0.166667	0.416667	0.067797	0.041667	0
2	0.111111	0.500000	0.050847	0.041667	0
3	0.083333	0.458333	0.084746	0.041667	0
4	0.194444	0.666667	0.067797	0.041667	0
...	...	...	...	...	...
145	0.666667	0.416667	0.711864	0.916667	2
146	0.555556	0.208333	0.677966	0.750000	2
147	0.611111	0.416667	0.711864	0.791667	2
148	0.527778	0.583333	0.745763	0.916667	2
149	0.444444	0.416667	0.694915	0.708333	2

150 rows × 5 columns

```
In [19]: # parallel coordinates 그리기(정규화 후)
parallel_coordinates(iris_scale, 'Species', color=('r','g','b'), alpha=0.5)
plt.show()
```



결측치 대체

```
In [44]: '''
`DataFrame.interpolate()` 함수는 누락된 데이터(결측치)를 보간하여 채우는 데
사용되는 Pandas의 함수입니다.
주어진 데이터프레임에서 NaN(Not a Number) 또는 NaT(Not a Time) 값이 있는 경우,
이를 선형 또는 다항식 등의 방법을 사용하여 주변 값들의 중간값으로 대체합니다.

이 함수의 주요 인자와 기능은 다음과 같습니다:
1. method (str, optional):
   - 보간 방법을 지정합니다.
   주요 보간 방법으로는 'linear', 'nearest', 'zero', 'slinear', 'quadratic',
   'cubic' 등이 있습니다. 기본값은 'linear'입니다.
```

2. `axis (int or str, optional):`
 - 보간을 수행할 축을 지정합니다. 기본값은 0입니다.
3. `limit (int, optional):`
 - 보간을 수행할 최대 연속 값의 개수를 제한합니다.
 - 기본값은 None으로, 모든 연속된 NaN 값을 보간합니다.
4. `inplace (bool, optional):`
 - 원본 데이터프레임을 변경할지 여부를 결정합니다.
 - 기본값은 False이며, 보간된 결과를 새로운 데이터프레임으로 반환합니다.
5. `limit_direction (str, optional):`
 - limit 인자가 사용될 때, 보간의 방향을 지정합니다.
 - 'forward', 'backward', 'both' 중 하나를 선택할 수 있습니다.
 - 기본값은 'forward'입니다.
6. `limit_area (str, optional):`
 - limit 인자가 사용될 때, 보간할 구역을 지정합니다.
 - 'inside', 'outside' 중 하나를 선택할 수 있습니다.
 - 기본값은 None으로, 구역 제한이 없습니다.
7. `downcast (optional):`
 - 보간된 데이터의 데이터 타입을 최소화하는 데 사용되는 옵션입니다.
 - 'infer', 'integer', 'signed', 'unsigned', 'float' 등을 선택할 수 있습니다.
8. `kwargs:`
 - 보간 방법에 따라 추가적인 인자를 전달할 수 있습니다.

주요 보간 방법(method)에 대한 설명은 다음과 같습니다:

- `linear`: 선형 보간을 수행합니다.
- `nearest`: 가장 가까운 값으로 보간합니다.
- `zero`: 0 값을 사용하여 보간합니다.
- `slinear`: 선형으로 보간하되, NaN 값을 무시합니다.
- `quadratic, cubic`: 2차 및 3차 다항식을 사용하여 보간합니다.

`DataFrame.interpolate()` 함수를 사용하면 결측치가 포함된 데이터프레임을 보간하여 데이터 손실을 최소화하고 분석에 적합한 데이터를 얻을 수 있습니다.

```
'''
```

```
import pandas as pd
import numpy as np
```

```
Image('1.png')
```

Out[44]:

$$y = y_1 + \frac{(x - x_1)(y_2 - y_1)}{(x_2 - x_1)}$$

In [45]:

```
'''
주요 보간 방법에 대해 자세히 설명하겠습니다:

1. Linear Interpolation (선형 보간):
- 선형 보간은 주어진 두 데이터 포인트 사이의 값들을 직선으로 연결하여 보간합니다.
- 두 데이터 포인트 $(x_1, y_1)$과 $(x_2, y_2)$ 사이의 보간된 값은 다음과 같이 계산됩니다:
  $$ y = y_1 + \frac{(x - x_1)(y_2 - y_1)}{(x_2 - x_1)} $$
- 선형 보간은 데이터가 선형적인 추세를 따를 때 주로 사용됩니다.

2. Nearest Neighbor Interpolation (최근접 이웃 보간):
- 최근접 이웃 보간은 보간할 위치와 가장 가까운 데이터 포인트의 값을 사용하여 보간합니다.
- 보간할 위치와 가장 가까운 데이터 포인트의 값을 그대로 사용합니다.
- 데이터가 이산적이고 불규칙할 때 사용될 수 있습니다.

3. Zero Interpolation (0 값 보간):
- 0 값 보간은 보간할 위치에서 0 값을 사용하여 보간합니다.
- 데이터 포인트 사이의 중간값을 0으로 채웁니다.
- 주로 데이터가 양수인 경우에 사용될 수 있습니다.

4. Spline Interpolation (스플라인 보간):
- 스플라인 보간은 보간할 데이터 포인트를 이용하여 구간별로 다항식을 정의하고, 이 다항식을 사용하여 보간합니다.
- 각 구간에서의 보간 다항식은 주어진 데이터 포인트를 지나며, 주변 구간의 다항식과 연속성과 미분의 연속성을 가지도록 설정됩니다.
- 스플라인 보간은 부드러운 곡선을 만들어야 할 때 사용됩니다. 주로 곡선이 부드럽고 데이터가 이산적이거나 불규칙할 때 사용됩니다.

5. Polynomial Interpolation (다항식 보간):
- 다항식 보간은 데이터 포인트를 지나는 하나의 다항식을 찾아서 보간합니다.
- 데이터 포인트를 모두 지나는 다항식을 찾기 위해 다항식 보간은 주어진 데이터 포인트의 개수보다 하나 작은 차수의 다항식을 사용
- 다항식 보간은 데이터가 다항식적인 추세를 따를 때 사용될 수 있습니다.

이러한 보간 방법들은 주어진 데이터의 특성에 따라 적합한 방법을 선택하여 사용해야 합니다. 데이터가 어떻게 분포되어 있는지와 보간된
'''

# 예시 데이터프레임 생성
data = {
    'A': [1, np.nan, 3, np.nan, 5],
    'B': [1, 2, np.nan, 4, 5]
}
```

```
df = pd.DataFrame(data)

# 선형 보간을 사용하여 결측값 채우기
df_interpolated = df.interpolate(method='linear', axis=0)

print("Original DataFrame:")
print(df)
print("\nDataFrame after interpolation:")
print(df_interpolated)
```

Original DataFrame:

	A	B
0	1.0	1.0
1	NaN	2.0
2	3.0	NaN
3	NaN	4.0
4	5.0	5.0

DataFrame after interpolation:

	A	B
0	1.0	1.0
1	2.0	2.0
2	3.0	3.0
3	4.0	4.0
4	5.0	5.0

```
In [21]: df2 = pd.DataFrame({'C1': [1, 2, np.nan, np.nan, 5],
                             'C2': [6, 8, 10, np.nan, 20]})
df2
```

```
Out[21]:
```

	C1	C2
0	1.0	6.0
1	2.0	8.0
2	NaN	10.0
3	NaN	NaN
4	5.0	20.0

```
In [22]: df2_mean = df2.fillna(df2.mean())  
df2_mean
```

```
Out[22]:
```

	C1	C2
0	1.000000	6.0
1	2.000000	8.0
2	2.666667	10.0
3	2.666667	11.0
4	5.000000	20.0

```
In [23]: df2_lin = df2.interpolate('linear')  
df2_lin
```

```
Out[23]:
```

	C1	C2
0	1.0	6.0
1	2.0	8.0
2	3.0	10.0
3	4.0	15.0
4	5.0	20.0

```
In [24]: df2_pol = df2.interpolate('polynomial',order=2)  
df2_pol
```



```
Out[24]:
```

	C1	C2
0	1.0	6.000000
1	2.0	8.000000
2	3.0	10.000000
3	4.0	13.894737
4	5.0	20.000000

이상치 탐지

LOF 예제

```
In [25]: '''
`LocalOutlierFactor()` 함수는 LOF(Local Outlier Factor)
이상치 탐지 알고리즘을 구현한 함수입니다.
이 함수는 데이터셋에서 이상치를 탐지하는 데 사용됩니다.
이 함수의 주요 인자들은 다음과 같습니다:

1. n_neighbors: 이웃의 수를 결정하는 매개변수입니다.
   LOF는 각 데이터 포인트의 이웃을 고려하여 이상치를 판별합니다.
   `n_neighbors` 값이 클수록 더 많은 이웃이 고려되므로
   이상치를 탐지하는 데 더 많은 컴퓨팅 자원이 필요합니다.
   일반적으로 20 이상의 값을 사용하는 것이 좋습니다.

2. contamination: 이상치로 간주할 데이터 포인트의 비율을 결정하는 매개변수입니다.
   예를 들어, 전체 데이터의 몇 퍼센트가 이상치로 간주될지를 지정합니다.
   이 값은 0과 1 사이의 값이어야 합니다. 기본값은 0.1입니다.

3. algorithm: 이웃을 계산하는 데 사용할 알고리즘을 지정하는 매개변수입니다.
   "auto", "ball_tree", "kd_tree", "brute" 중 하나를 선택할 수 있습니다.
   기본값은 "auto"로, 자동으로 가장 적합한 알고리즘을 선택합니다.

4. leaf_size: BallTree 또는 KDTree를 구성하는 잎(leaf)의 크기입니다.
   더 작은 leaf_size는 더 나은 성능을 제공하지만
   메모리 사용량이 증가할 수 있습니다.

5. metric: 거리 측정 방법을 지정하는 매개변수입니다.
```

"euclidean", "manhattan", "chebyshev", "minkowski", "wminkowski",
 "seuclidean", "mahalanobis" 등을 선택할 수 있습니다.

6. `p`: Minkowski 거리를 계산하는 데 사용되는 파라미터입니다.
 기본값은 2로, 일반적으로 유클리디안 거리를 의미합니다.

``LocalOutlierFactor()`` 함수의 주요 특징과 사용법은 다음과 같습니다:

1. **LOF(Local Outlier Factor) 알고리즘**: LOF 알고리즘은 이상치를 탐지하기 위한 비지도 학습 알고리즘 중 하나입니다.
 이 알고리즘은 각 데이터 포인트의 이웃들과 그들의 밀도를 고려하여 이상치를 식별합니다.
 이상치는 주변 이웃들보다 훨씬 더 드문 밀도를 갖는 포인트로 간주됩니다.
2. **거리 측정 방법 지원**: ``metric`` 매개변수를 사용하여 다양한 거리 측정 방법을 지정할 수 있습니다. 이는 데이터의 특성에 따라 적합한 거리 측정 방법을 선택할 수 있는 유연성을 제공합니다.
3. **이상치 비율 지정**: ``contamination`` 매개변수를 사용하여 데이터셋에서 이상치로 간주할 비율을 지정할 수 있습니다. 이 값은 전체 데이터 포인트 중 이상치로 간주될 비율을 나타내며, 알고리즘이 이상치를 판별하는 데 중요한 역할을 합니다.
4. **사용법**: ``LocalOutlierFactor()`` 함수는 다음과 같이 사용됩니다.

```
from sklearn.neighbors import LocalOutlierFactor
```

```
# LOF 객체 생성
```

```
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.1)
```

```
# 데이터에 대한 이상치 탐지
```

```
outlier_scores = lof.fit_predict(X)
```

이상치 탐지 결과는 ``fit_predict()`` 메서드를 호출하여 얻을 수 있습니다.

이 메서드는 각 데이터 포인트에 대한 이상치 점수를 반환하며,

이 점수는 각 데이터 포인트가 이상치로 간주될 가능성을 나타냅니다.

이상치 점수가 음수인 경우 이상치로 간주됩니다.

5. **결과 해석**: 이상치 점수를 기반으로 이상치를 해석할 수 있습니다.

일반적으로 절대적인 이상치 점수의 임계값을 정의하거나

상위 백분위수를 사용하여 상위 몇 퍼센트의 데이터 포인트를 이상치로 간주할 수 있습니다.

이상치 탐지에는 데이터의 특성과 도메인 지식을 고려하여

적절한 매개변수를 선택하고 결과를 해석하는 것이 중요합니다.

```
...
```

```
...
```

``LocalOutlierFactor()`` 함수의 ``fit_predict()`` 메서드는
 각 데이터 포인트에 대한 이상치 점수를 계산하고,

이상치 여부를 예측한 결과를 반환합니다.
 이상치 점수는 각 데이터 포인트의 LOF 값으로 나타내며,
 LOF 값이 음수인 경우 해당 데이터 포인트는 이상치로 간주됩니다.

`fit\_predict()` 메서드의 반환값은 다음과 같습니다:

1. 이상치 점수 배열: LOF 값은 각 데이터 포인트에 대한 이상치 정도를 나타냅니다.
 배열의 각 원소는 해당 데이터 포인트의 LOF 값에 해당합니다.
 이상치 점수가 음수인 경우 이상치로 간주됩니다.
 2. 이상치 예측 결과: 이상치 점수를 기반으로 한 예측 결과가 반환됩니다.
 일반적으로 이상치 점수가 음수인 데이터 포인트는 이상치로 간주되므로,
 이상치로 예측된 데이터 포인트는 -1의 값을 갖습니다.
 반면 이상치가 아닌 정상적인 데이터 포인트는 1의 값을 갖습니다.
- 따라서 `fit\_predict()` 메서드의 반환값은 LOF 값으로 이루어진 배열이며,
 이를 통해 각 데이터 포인트의 이상치 여부를 예측할 수 있습니다.

`LocalOutlierFactor()` 함수는 여러 속성을 제공하여
 사용자가 이상치 탐지 결과를 분석하고 해석할 수 있도록 도와줍니다.
 주요 속성은 다음과 같습니다:

1. `negative_outlier_factor_`: 이 속성은 각 데이터 포인트의 LOF(Local Outlier Factor) 값을
 포함하는 배열을 제공합니다.
 LOF 값은 해당 데이터 포인트가 이상치로 간주될 가능성을 나타내는 지표입니다.
 값이 음수인 경우 이상치로 간주됩니다.
2. `n_neighbors_`: LOF 계산에 사용된 이웃의 수를 나타내는 속성입니다.
 이웃의 수는 `LocalOutlierFactor()` 함수의 `n\_neighbors` 매개변수에 의해 결정됩니다.
3. `offset_`: 이 속성은 이상치 점수에 추가할 오프셋을 나타냅니다.
 이 값은 `contamination` 매개변수를 기반으로 LOF 알고리즘에 의해 결정됩니다.

이러한 속성들을 통해 사용자는 LOF 알고리즘을 통해 계산된
 이상치 점수 및 다른 중요한 정보를 확인하고,
 이를 기반으로 이상치를 식별하고 해석할 수 있습니다.
 ...

```
from sklearn.neighbors import LocalOutlierFactor
import numpy as np
```

```
# 예시 데이터 생성
```

```
X = np.array([[1, 1], [2, 2], [3, 3], [10, 10]])
```

```
# LOF 모델 생성 및 fitting
```

```
lof_model = LocalOutlierFactor(n_neighbors=2)
outlier_scores = lof_model.fit_predict(X)
```

```
# 이상치 점수 출력  
print("Outlier scores:", outlier_scores)
```

Outlier scores: [1 1 1 -1]

```
In [26]: from sklearn.neighbors import LocalOutlierFactor
```

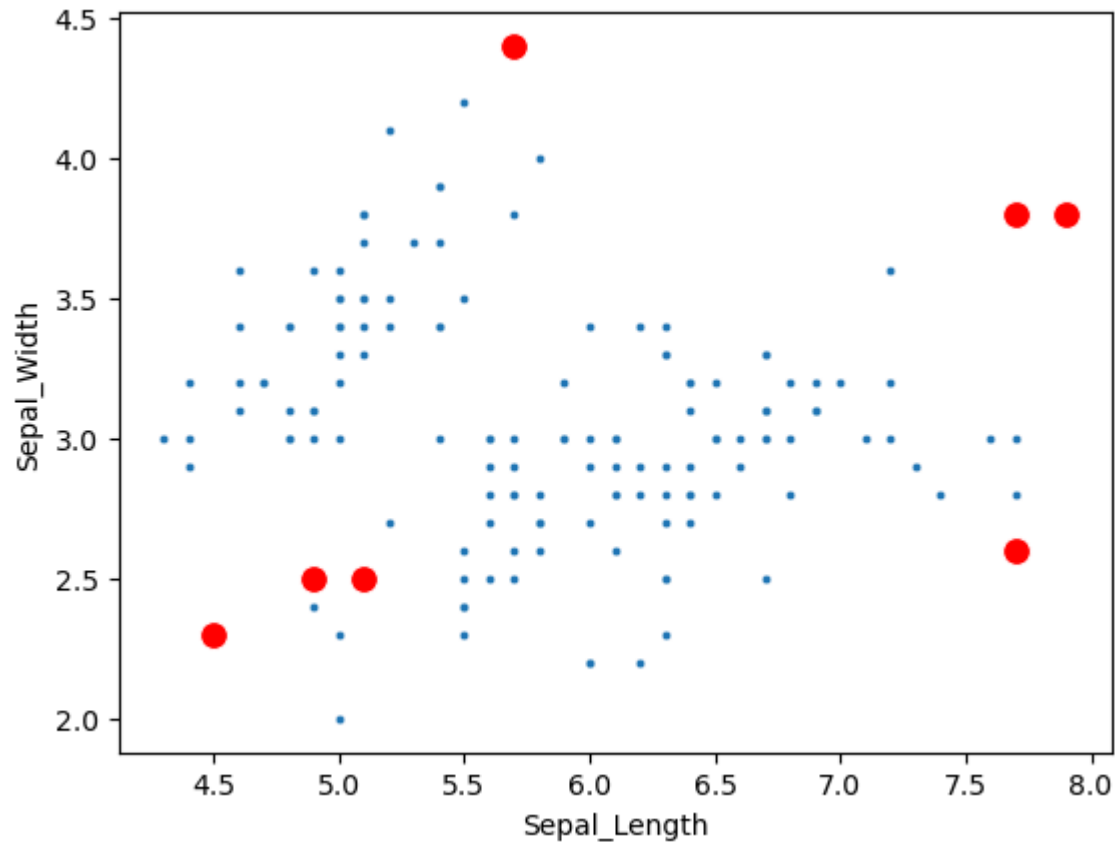
```
In [27]: clf = LocalOutlierFactor(n_neighbors=20, contamination='auto')  
y_pred = clf.fit_predict(iris) #1(normal), -1(outlier)  
X_scores = clf.negative_outlier_factor_  
X_scores
```

```
Out[27]: array([-0.97441839, -1.01339844, -1.02439063, -1.05593645, -0.95923775,
-1.15241648, -1.02028254, -0.96486836, -1.19431941, -0.96590417,
-1.03930799, -0.96287672, -1.02987361, -1.32568723, -1.48020323,
-1.67187225, -1.15306178, -0.97536351, -1.24180918, -1.00311027,
-1.02534917, -1.0023451 , -1.27368435, -0.99320982, -1.07242318,
-0.98488988, -0.97517153, -0.97921234, -0.97506634, -1.01723168,
-0.99950206, -0.99491375, -1.21162996, -1.36588204, -0.9700006 ,
-0.97752859, -1.07916688, -0.94531486, -1.16500566, -0.97714549,
-0.95284631, -1.77758216, -1.13515903, -1.01241984, -1.13644094,
-1.02786816, -1.00537484, -1.04856947, -1.02026449, -0.96826491,
-1.18214688, -1.02636039, -1.14092364, -1.03165777, -1.0332014 ,
-0.94310421, -1.0331743 , -1.42992091, -1.03411943, -1.01956718,
-1.41924458, -0.94251081, -1.06125085, -0.99454865, -1.0247867 ,
-1.04425952, -0.98497548, -0.98420379, -1.09942177, -0.99566277,
-1.06349891, -0.96342874, -1.04055533, -0.9888635 , -0.98414331,
-1.02309519, -1.06238865, -1.10391296, -0.96938876, -1.10347443,
-1.05444577, -1.1068384 , -0.98589844, -1.06894864, -1.02299891,
-1.00872849, -1.0396374 , -1.04085716, -0.97775553, -0.99566277,
-0.98044172, -0.99613477, -0.9882629 , -1.39847107, -0.99178185,
-0.96221162, -0.97409576, -0.94204665, -1.50815082, -0.98265086,
-1.10083833, -1.06208123, -1.04930505, -0.96975015, -0.98331163,
-1.38911722, -1.61899825, -1.25486778, -1.04748027, -1.21030966,
-0.98023848, -0.97433522, -0.96697728, -1.12772445, -1.08644689,
-0.96713926, -0.96959211, -1.56142134, -1.6119805 , -1.16742223,
-1.01791855, -1.152035 , -1.47970105, -1.02066243, -0.98537885,
-1.12778552, -1.04067099, -1.01263349, -0.96080968, -1.14399113,
-1.23245056, -1.53590203, -0.96766854, -1.02624864, -1.07294907,
-1.29524167, -0.99797575, -0.9750274 , -1.0541535 , -0.96860791,
-0.99045377, -1.00598916, -1.06208123, -1.02017133, -1.02530128,
-0.96570202, -1.02670516, -0.97600688, -0.99093352, -1.01823187])
```

```
In [28]: y_pred
```

```
Out[28]: array([ 1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1,  1,
                1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,
                1,  1,  1,  1,  1,  1,  1, -1,  1,  1,  1,  1,  1,  1,  1,  1,  1,
                1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,
                1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,
                1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1,  1,  1,  1,
                1,  1,  1,  1, -1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1, -1,
                1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1,  1,  1,  1,  1,
                1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1])
```

```
In [29]: sns.scatterplot(data=iris, x='Sepal_Length', y='Sepal_Width', s=10)
sns.scatterplot(data=iris.loc[y_pred!=-1], x='Sepal_Length', y='Sepal_Width',
                s=100, color='red')
plt.show()
```



데이터 변환

Decimal Scaling

```
In [30]: '''
이 함수는 입력으로 데이터프레임(df)을 받아들이어서 각 열(column)의 값을
각 열의 최댓값의 자릿수에 따라서 조정합니다. 함수는 다음과 같은 과정을 거칩니다:

1. 입력된 데이터프레임(df)을 복사하여 dec_df 변수에 저장합니다.
2. for 루프를 통해 데이터프레임의 각 열에 대해서 다음을 수행합니다:
   - 열의 최댓값을 구합니다.
   - 최댓값의 절대값을 문자열로 변환하여 그 길이를 구합니다.
   - 각 열의 값들을 10의 (최댓값의 자릿수) 제곱으로 나누어 줌으로써 값을 조정합니다.
     이는 해당 열의 값을 최댓값의 자릿수에 맞게 스케일링하여 줍니다.
3. 조정된 데이터프레임(dec_df)을 반환합니다.

이 함수는 데이터프레임의 각 열의 값을 해당 열의 최댓값의 자릿수에 따라 스케일링하여 줌으로써,
값들의 크기 차이를 줄여주는 역할을 합니다.
이는 데이터 분석 및 모델링 과정에서
각 특성(feature)의 중요성을 공정하게 고려하고자 할 때 유용합니다.
'''

def Dec_scale(df):
    dec_df = df.copy()
    for x in df:
        p = df[x].astype("int32").max()
        q = len(str(abs(p)))
        dec_df[x] = df[x]/10**q
    return dec_df
```

```
In [31]: Dec_scale(fastfood)
```

Out[31]:

	Popeyes	Wendys	McDonalds	Hardees	Jack in Box
0	0.150	0.173	0.198	0.158	0.289
1	0.197	0.095	0.131	0.195	0.273
2	0.186	0.154	0.158	0.177	0.254
3	0.166	0.104	0.123	0.147	0.282
4	0.196	0.183	0.139	0.181	0.235
...	...	...	...	...	...
95	0.195	0.138	0.145	0.235	0.272
96	0.213	0.154	0.178	0.218	0.261
97	0.135	0.178	0.133	0.147	0.271
98	0.136	0.152	0.124	0.154	0.263
99	0.205	0.119	0.180	0.202	0.265

100 rows × 5 columns

Power 변환

```
In [32]: df = pd.read_csv("creditcard.csv")
df1 = df.copy()
df1
```


Out[32]:

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_
0	40.900749	0.818182	95.40	0.00	95.40	0.000000	
1	3202.467416	0.909091	0.00	0.00	0.00	6442.945483	
2	2495.148862	1.000000	773.17	773.17	0.00	0.000000	
3	1666.670542	0.636364	1499.00	1499.00	0.00	205.788017	
4	817.714335	1.000000	16.00	16.00	0.00	0.000000	
...	...	...	...	...	...	...	...
8945	28.493517	1.000000	291.12	0.00	291.12	0.000000	
8946	19.183215	1.000000	300.00	0.00	300.00	0.000000	
8947	23.398673	0.833333	144.40	0.00	144.40	0.000000	
8948	13.457564	0.833333	0.00	0.00	0.00	36.558778	
8949	372.708075	0.666667	1093.25	1093.25	0.00	127.040008	

8950 rows × 17 columns



In [33]:

```
'''
`seaborn`의 `kdeplot()` 함수는 커널 밀도 추정(kernel density estimation)을 사용하여
1차원 또는 2차원 데이터의 분포를 시각화하는 데 사용됩니다.
주어진 데이터의 분포를 부드러운 곡선 형태로 그려주어 데이터의 밀도 추정을 제공합니다.
`kdeplot()` 함수의 주요 인자들은 다음과 같습니다:

1. data: 시각화할 데이터를 포함하는 1차원 또는 2차원 배열 또는 데이터프레임입니다.
2. x, y: 데이터의 x축 및 y축에 대응하는 데이터 변수입니다.
   1차원 데이터의 경우 x만 지정하면 됩니다.
3. shade: 밀도 곡선 아래에 음영을 추가할지 여부를 나타내는 부울(boolean) 값입니다.
   기본값은 True로 설정되어 있습니다.
4. vertical: 밀도 추정 그래프를 수직 방향으로 플로팅할지 여부를 나타내는
   부울(boolean) 값입니다. 기본값은 False로 설정되어 있습니다.
5. kernel: 사용할 커널 함수의 종류를 지정하는 문자열입니다.
```

기본값은 "gau"로, 가우시안 커널을 사용합니다.
 다른 옵션으로 "cos", "biw", "epa", "tri", "triw" 등이 있습니다.

6. **bw**: 커널의 대역폭(bandwidth)을 지정하는 매개변수입니다.
 작은 값은 부드러운 추정을 생성하고, 큰 값은 더 거친 추정을 생성합니다.
 기본값은 "scott"로, Scott's Rule에 따라 대역폭이 자동으로 선택됩니다.
 다른 옵션으로 "silverman" 등이 있습니다.

7. **gridsize**: 등고선을 그릴 그리드의 크기를 지정하는 정수입니다. 기본값은 100입니다.

8. **cumulative**: 누적 밀도 함수(cumulative density function)를 플로팅할지 여부를 나타내는 부울(boolean) 값입니다. 기본값은 False로 설정되어 있습니다.

9. **common_norm**: 여러 데이터셋을 함께 플로팅할 때 정규화(normalization)를 사용할지 여부를 나타내는 부울(boolean) 값입니다. 기본값은 True로 설정되어 있습니다.

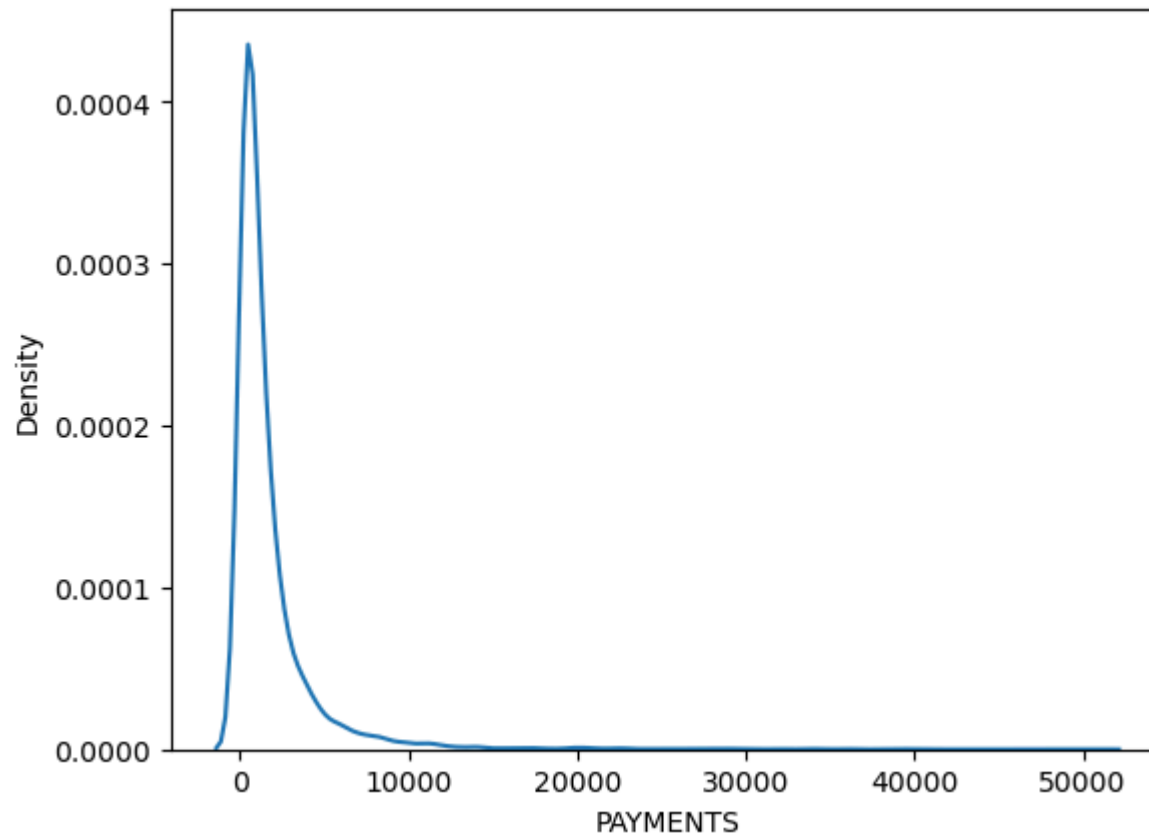
10. **color**: 밀도 곡선의 색상을 지정하는 문자열이나 색상 코드입니다.

11. **linewidth, linestyle, alpha**: 밀도 곡선의 선의 너비, 스타일 및 투명도를 제어하는 매개변수입니다.

``kdeplot()`` 함수를 사용하여 데이터의 분포를 시각화할 때,
 위의 인자들을 조정하여 원하는 시각화 결과를 얻을 수 있습니다.

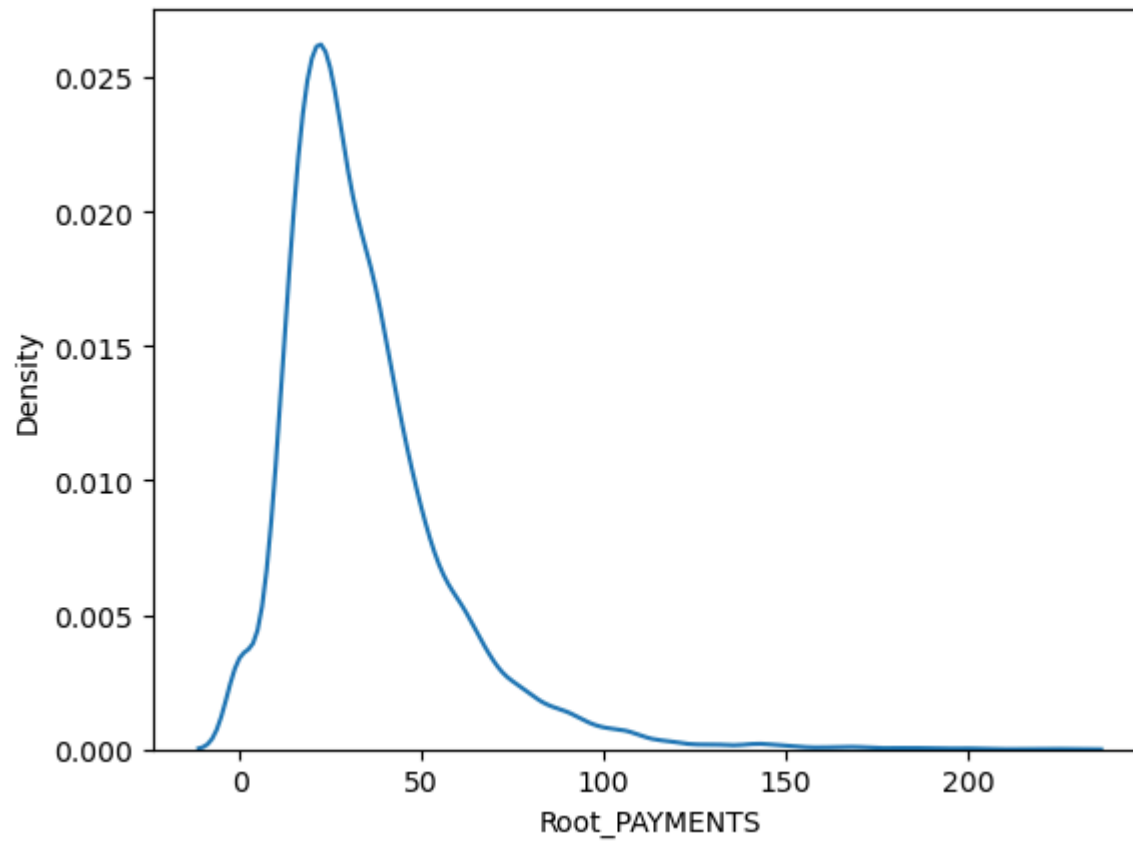
```
'''
sns.kdeplot(data=df1, x='PAYMENTS')
```

```
Out[33]: <Axes: xlabel='PAYMENTS', ylabel='Density'>
```



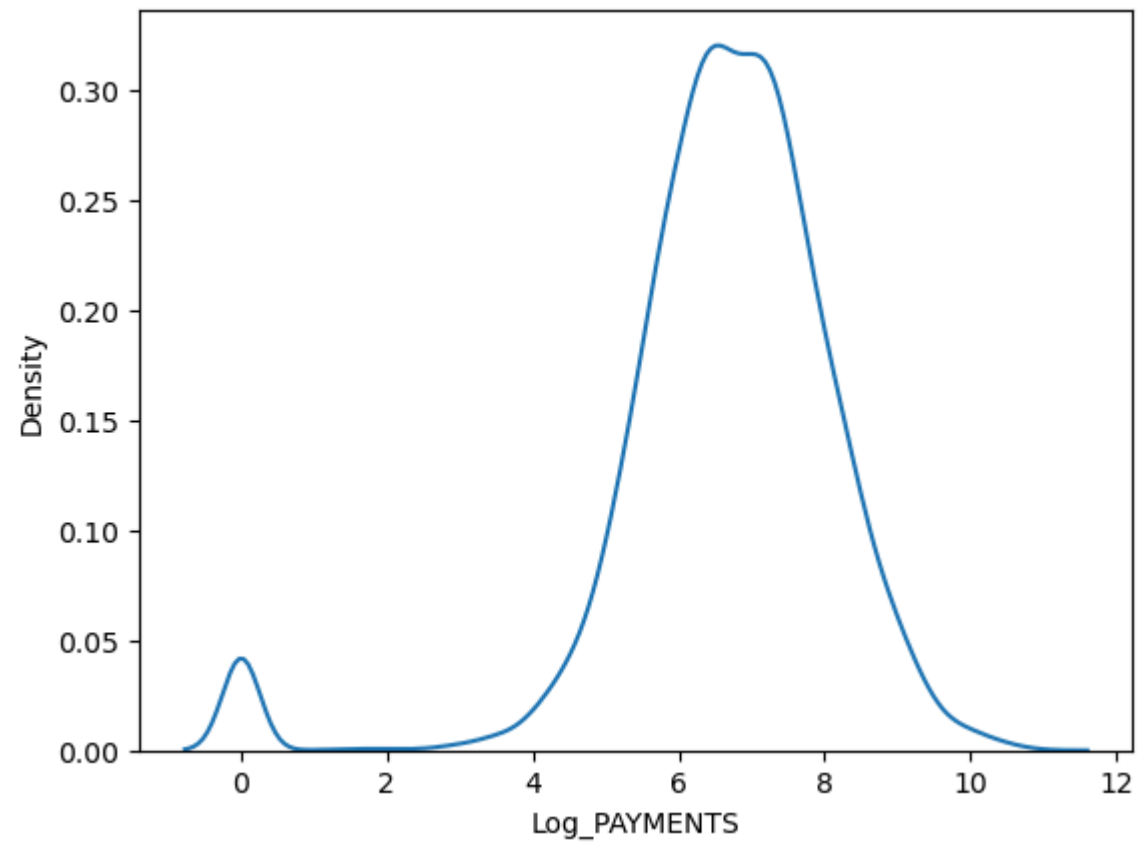
```
In [34]: df1['Root_PAYMENTS'] = np.sqrt(df1['PAYMENTS'])  
sns.kdeplot(data=df1, x='Root_PAYMENTS')
```

```
Out[34]: <Axes: xlabel='Root_PAYMENTS', ylabel='Density'>
```



```
In [35]: df1['Log_PAYMENTS'] = np.log(1+df1['PAYMENTS'])  
sns.kdeplot(data=df1, x='Log_PAYMENTS')
```

```
Out[35]: <Axes: xlabel='Log_PAYMENTS', ylabel='Density'>
```



In []: